Module 9 — Commands, Hooks & Reusable Workflows

Claude Code Bootcamp · Day 1 · Block 9 of 10

Promise

In 22 minutes you will:

1. Read an existing skill (`skills/code-review/SKILL.md`) as a worked example.
2. Author **your own** `SKILL.md` following the contract.
3. Invoke your new skill on a real task and verify it earns its rent.

Why this matters

- A skill is the unit of *carry-over*. It is the prompt that survives leaving this workshop.
- Skills make Claude consistent across projects, teams, and days. Without skills you re-invent each prompt.
- The 10 skills shipped with this repo are the durable artifact you take home — but the one *you* write is the skill you'll actually use first.

Concepts

- **Skill = directory** at `skills/<kebab-name>/SKILL.md`.
- **Frontmatter**: `name`, `description`. Both required.
- **Body**: 6 H2 sections — Purpose · When to use · Body · Inputs · Outputs · Worked example.
- **Project-agnostic**: a skill that mentions your repo's specific filenames is broken. Reference shapes, not paths.
- **The contract**: see `specs/001-bootcamp-course-materials/contracts/skill.contract.md`.

Live demo flow

1. Instructor opens `skills/code-review/SKILL.md`. Reads each H2 header aloud.
2. Picks a real task from earlier today — e.g., the BoN scoring loop.
3. Drafts a new skill `score-candidates` in 4 minutes following the same shape.
4. Invokes it: *"Use the `score-candidates` skill to evaluate these three diffs."*
5. Class watches Claude produce structured output that matches the skill's "Outputs" section.

Mini project

Author one new skill of your own. Pick a workflow you actually want to repeat:

- `commit-and-pr` — generate Conventional Commits + PR text from a diff
- `screenshot-diff` — compare a render to a wireframe and patch the gap
- `regression-postmortem` — write a one-page postmortem from a failing test
- *(Any other repeatable workflow you noticed today.)*

Step-by-step lab

1. Open `skills/code-review/SKILL.md` in one pane and `specs/001-bootcamp-course-materials/contracts/skill.contract.md` in another.
2. Pick a name (kebab-case). Create `module-09/skill/SKILL.md`.
3. Fill the frontmatter, then each H2 section in order.
4. Self-check: does the file mention any path or filename specific to this workshop? If yes, generalise it.
5. Invoke your skill on a real input from earlier today (a diff, a render, a candidate set).
6. If the output matches your "Outputs" section: ship it. If not: tighten "Body" and re-run.

Suggested Claude Code prompts

```
DRAFT THE SKILL
I want a Claude Skill called `<your-name>` that <one-sentence purpose>.
Following the contract at specs/001-bootcamp-course-materials/contracts/skill.contract.md,
draft the SKILL.md. Body must be project-agnostic — no references to specific
filenames or framework versions. Worked example must be runnable as-is.
```

```
INVOKE THE SKILL
Use the `<your-name>` skill at module-09/skill/SKILL.md.
Inputs: <attach or paste the input>.
Produce the output exactly as the skill's "Outputs" section specifies.
```

Deliverable checklist

- [ ] `module-09/skill/SKILL.md` exists with valid frontmatter (`name`, `description`).
- [ ] All 6 body H2 sections present in order.
- [ ] No project-specific paths, filenames, or version numbers in the body.
- [ ] One real invocation captured in `module-09/invocation.md` (the prompt + the output).

Definition of done

✅ Skill validates against the contract · ✅ Project-agnostic · ✅ Real invocation produced output that matches the "Outputs" section.

Review checkpoint

Pair (60 s each):

1. Read partner's skill. Could you drop it into a *different* repo and have it work?
2. Identify one sentence in "Body" that is too vague to act on.

Common mistakes

- Skill body that says "do a good job" — meaningless. Be operational.
- Embedding repo-specific paths or framework versions. The skill won't carry over.
- Skipping "Worked example". Reviewers can't tell whether the skill works without it.
- Authoring 5 skills in 22 minutes. Author 1 well; carry the others home as homework.

Instructor notes

- 5 / 5 / 10 / 2 split.
- Open `skills/code-review/SKILL.md` live; many students will copy its shape verbatim, which is fine.
- Reinforce FR-018: project-agnostic. The validator will reject skills that fail the carry-over test.
- If short, drop the second invocation; one is enough.

Transition to next module

We have skills, code, tests, branches, docs. The last 18 minutes ask the hardest question: would you put any of this in production tomorrow?

Next: Module 10 — Production Readiness.